Contents

PaisaPilot — Full-Phase Technical Requirements Document (TRD) **2**
1. Purpose and Audience . . . 2
2. System Scope and Boundaries . . . 3
3. Recommended Technology Stack . . . 4
4. High-Level Architecture . . . 6
5. Cross-Cutting Engineering Requirements . . . 8
6. Data Model and Database Requirements . . . 11
7. API Technical Contract . . . 15
8. Phase-by-Phase Implementation Plan . . . 18
Phase 0 — Repository Audit and Technical Baseline . . . 18
Phase 1 — Project Setup and Developer Experience . . . 19
Phase 2 — Authentication, Authorization, and Account Lifecycle . . . 19
Phase 3 — Transaction, Category, and Payment-Source Core . . . 20
Phase 4 — Statement Upload, Parsing, Preview, and Commit . . . 20
Phase 5 — Dashboard, Analytics, and Reports . . . 20
Phase 6 — AI-Assisted Categorization . . . 21
Phase 7 — Grounded AI Finance Assistant . . . 21
Phase 8 — Budgets, Thresholds, and Notifications . . . 22
Phase 9 — Recurring Payment Detection . . . 22
Phase 10 — Cash-Flow Forecasting . . . 22
Phase 11 — Export, Deletion, Retention, and Privacy Controls . . . 23
Phase 12 — Production Hardening and Release Readiness . . . 23
Phase 13 — Optional Authorized Bank-Data Integration . . . 24
9. Statement Import Engineering Detail . . . 24
10. Analytics Definitions . . . 26
11. AI Architecture, Safety, and Reliability . . . 27
12. Security Requirements . . . 28
13. Privacy, Compliance, and Responsible Use . . . 30
14. Non-Functional Requirements (NFRs) . . . 30
15. Testing Strategy . . . 31
16. Deployment and Operations . . . 33
17. AI Coding Agent / Developer Execution Protocol . . . 34
18. Definition of Done (Global) . . . 35
19. Release Checklist . . . 35
20. Open Technical Decisions . . . 36
21. Suggested Architecture Decision Records . . . 36
22. Recommended Implementation Order . . . 37
23. Final Technical Direction . . . 37

PaisaPilot — Full-Phase Technical Requirements Document (TRD)

Product: PaisaPilot
Document type: Technical Requirements Document
Version: 1.0
Status: Implementation blueprint
Target market: India, initial release
Primary bank-statement target: State Bank of India (SBI), through user-uploaded statements
Application type: Responsive web application / Progressive Web App-ready architecture
Companion document: `prd.md` (product requirements and acceptance checkpoints)

---

1. Purpose and Audience

This TRD converts PaisaPilot's product requirements into a technical build plan. It is intended for developers, reviewers, QA engineers, deployment owners, and AI coding agents. Requirements are grouped into phases so implementation can proceed incrementally, with concrete outputs and exit criteria for every phase.

This document is not a claim that any bank API, payment rail, AI provider, or regulatory permission is already available. Provider eligibility, contracts, consent requirements, and applicable legal obligations must be verified before a production integration is enabled.

1.1 Key engineering principles

1. **Correctness before intelligence:** transaction ingestion and server-side calculations must be reliable before AI insights are enabled.
2. **Private by default:** financial data is accessible only to the authenticated owner and explicitly authorized backend services.
3. **Deterministic money math:** use integer paise or exact decimal arithmetic; never use binary floating-point for authoritative monetary totals.
4. **Explainable AI:** AI may explain verified calculations and propose classifications, but it must not invent transactions, balances, or financial facts.
5. **Idempotent workflows:** retried imports, webhooks, and background jobs must not duplicate committed effects.
6. **Small, reviewable changes:** work one phase at a time, preserve working code, and test before marking a checkpoint complete.
7. **Secure failure:** errors must not expose secrets, statement content, session tokens, stack traces, or another user's data.
8. **No fake integrations:** unavailable bank connections, notifications, or AI features must be clearly shown as unavailable rather than simulated as

real.

---

2. System Scope and Boundaries

2.1 Initial release capabilities

- Registration, authentication, session management, and account recovery.
- Manual income and expense entry, editing, deletion, search, filtering, and pagination.
- Category management and payment-source labels.
- CSV statement import and specifically tested SBI PDF statement import formats.
- Import preview, row validation, duplicate detection, status tracking, retry, and error reporting.
- Income, expense, net cash-flow, category, and period analytics.
- Monthly budgets, threshold notifications, recurring-payment candidates, and reminders.
- AI-assisted transaction categorization and grounded financial Q&A.
- Basic cash-flow forecasts only after sufficient reliable historical data exists.
- User data export and deletion workflows.
- Operational logging, monitoring, backups, rate limiting, and security testing.

2.2 Explicit non-goals for the initial release

- Initiating bank transfers, UPI payments, bill payments, or card payments.
- Collecting bank login passwords, OTPs, PINs, CVVs, or UPI PINs.
- Claiming real-time bank balances from partial or stale statement imports.
- Unverified direct SBI API access, screen scraping, or browser automation of bank portals.
- Guaranteed forecasts, credit decisions, tax determinations, or regulated investment recommendations.
- Multi-tenant business accounting, shared family accounts, or collaborative finance management unless added to a later approved scope.

2.3 User-facing data freshness labels

Every account/analytics page must distinguish between: - **Imported data:** last transaction date and most recent successful import time, where known. - **Manually entered data:** user-provided records, not independently verified by a bank. - **Forecast:** a model-based estimate with its assumptions and historical period disclosed. - **Live connection:** permitted only after a real, verified authorized provider is integrated and its connection status is reliable.

Do not label an imported statement's last known balance as a live balance.

3. Recommended Technology Stack

3.1 Application stack

Layer	Default choice	Technical notes
Language	TypeScript, strict mode	Share types where appropriate; runtime validation is still mandatory.
Web application	Next.js App Router	Use Server Components by default; use Client Components only for interactive UI.
UI	React + Tailwind CSS	Responsive, accessible components; consistent design tokens.
Component library	shadcn/ui or an equivalent maintained component system	Avoid duplicating primitive controls without reason.
Icons	Lucide	Consistent icon set.
Charts	Recharts	Charts must have accessible labels and useful empty states.
API	Next.js Route Handlers	Modular service layer; don't put business logic directly in route files.
Database	MongoDB Atlas	Use Mongoose or official MongoDB Node.js driver consistently; do not mix data access patterns without a reason.
Validation	Zod	Validate untrusted inputs at runtime, including environment variables.
Authentication	Established auth library/provider	Select based on current Next.js compatibility, secure sessions, account recovery, and operational needs.

Layer	Default choice	Technical notes
AI integration	Server-only provider adapter	Provider-agnostic interface, structured output, bounded context, timeouts, and budget limits.
Background jobs	Durable queue + Node.js worker	Use for imports, bulk categorization, recurring analysis, scheduled alerts, and large exports.
File storage	Private object storage or private temporary storage	Random object keys, short retention, restricted access, and lifecycle cleanup.
Unit/integration tests	Vitest + relevant test utilities	Test money math, access control, parser logic, and services.
UI tests	React Testing Library	Focus on user-visible behavior.
End-to-end tests	Playwright	Test critical user journeys and authorization boundaries.
CI/CD	GitHub Actions or equivalent	Lint, type-check, test, build, dependency checks, and migration validation.
Monitoring	Structured logs + error tracking + uptime checks	Scrub personally identifiable and financial data from telemetry.

3.2 Architecture decision

Begin with a modular monolith: a single Next.js application with clear modules and a separate worker process for long-running tasks. Avoid microservices in the MVP. A separate Express API is optional only if deployment topology, external clients, or scaling needs justify it later.

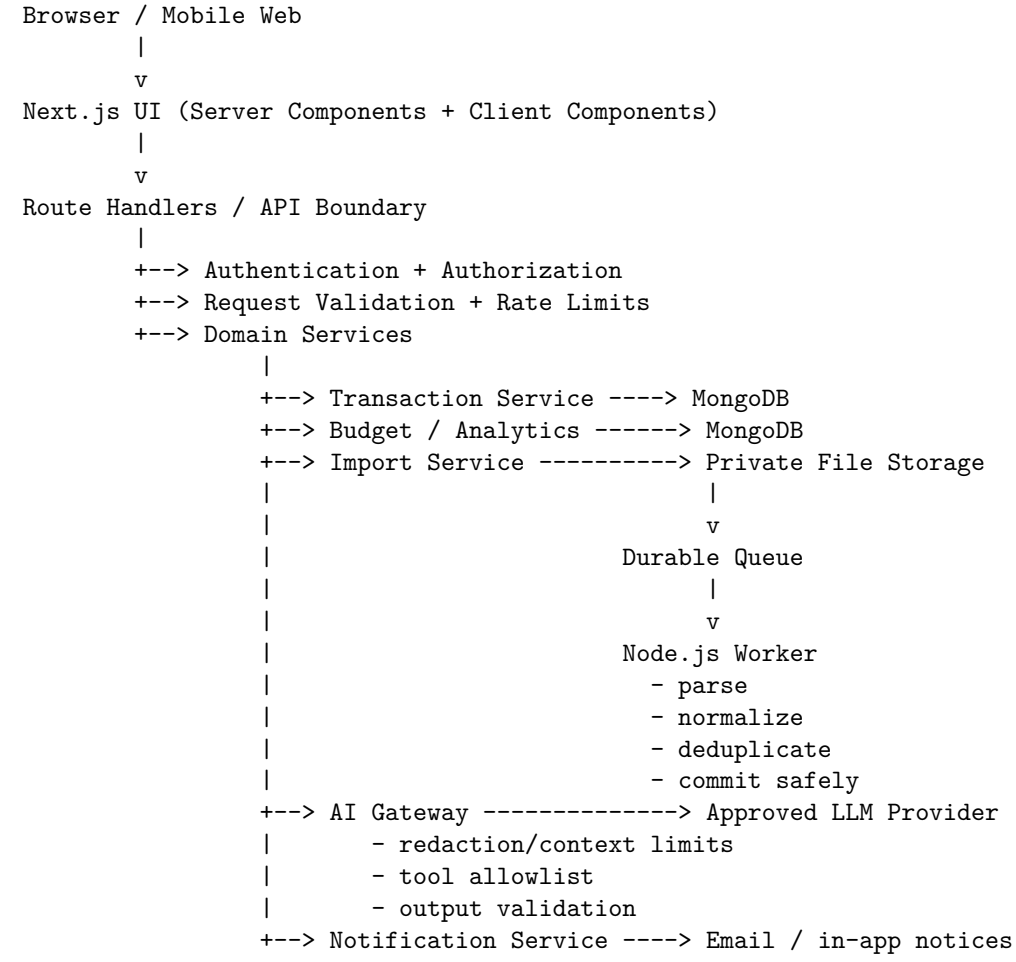
3.3 Version policy

- Pin supported dependency versions using the package manager lockfile.
- Do not upgrade Next.js, React, the database driver, or authentication packages in the middle of a feature phase without a clear need and regression test plan.

- Keep staging and production configuration separate.
- Regularly review dependency advisories and remove unused packages.

---

4. High-Level Architecture



4.1 Trust boundaries

- Browser inputs and uploaded files are untrusted.
- The API boundary is responsible for authentication, authorization, schema validation, rate limiting, and safe error mapping.
- Domain services enforce business rules and must not trust UI calculations.

- Workers must validate job ownership and payloads independently; queue messages are not proof of authorization.
- The AI provider receives only the minimum data required for the request and must never receive credentials or unrestricted database access.
- Logs and monitoring systems are not a place to store financial records or uploaded statement content.

4.2 Suggested code organization

```
src/
  app/
    (public)/
    (auth)/
    (dashboard)/
    api/
      health/route.ts
      transactions/route.ts
      transactions/[id]/route.ts
      imports/route.ts
      imports/[id]/route.ts
      budgets/route.ts
      analytics/route.ts
      ai/categorize/route.ts
      ai/chat/route.ts
  components/
    ui/
    layout/
    dashboard/
    transactions/
    imports/
    budgets/
    charts/
  modules/
    auth/
    users/
    transactions/
    categories/
    imports/
    analytics/
    budgets/
    recurring/
    forecasting/
    ai/
    notifications/
    audit/
  lib/
```

```
    config/
    db/
    auth/
    validation/
    errors/
    logging/
    rate-limit/
    money/
    dates/
  models/              # if using Mongoose; otherwise use repository schemas
  workers/
    import-worker.ts
    scheduled-jobs.ts
  tests/
    unit/
    integration/
    e2e/
```

Adapt this structure to an existing repository instead of forcing a full rewrite. Avoid duplicate `models`, `modules`, or `services` directories that hold the same responsibility.

---

5. Cross-Cutting Engineering Requirements

5.1 Environment configuration

- Parse environment variables once in a server-only configuration module.
- Fail fast at startup for required missing configuration; never print secret values.
- Maintain `.env.example` with placeholders only.
- Keep AI provider keys, database credentials, session secrets, email credentials, and storage credentials server-side.
- Never prefix secret server configuration with a client-exposed variable prefix.
- Use separate secrets for development, staging, and production.

Required variables depend on selected providers but will generally include:

```
NODE_ENV=
APP_BASE_URL=
MONGODB_URI=
AUTH_SECRET=
AI_PROVIDER=
AI_API_KEY=
PRIVATE_STORAGE_BUCKET=
EMAIL_PROVIDER_API_KEY=
```

This is illustrative; include only variables actually used by the implementation. Provide safe feature flags for optional modules. Never commit real values.

5.2 API conventions

- Use JSON for normal API requests and responses, unless a file-upload endpoint requires multipart form data.
- Validate all inputs with schemas on the server.
- Use consistent HTTP status codes and stable machine-readable error codes.
- Use pagination for all potentially large collections.
- Set maximum request/body/file sizes.
- Use ISO 8601 timestamps on the wire, with a documented timezone strategy.
- Include a request/correlation ID in server logs and safe error responses.
- Do not expose internal MongoDB errors, raw provider responses, stack traces, or secrets.

Suggested response shape:

```
{
  "data": {},
  "meta": { "requestId": "server-generated-id" }
}
```

Suggested error shape:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Check the highlighted fields and try again.",
    "fieldErrors": {}
  },
  "meta": { "requestId": "server-generated-id" }
}
```

Do not return `data: null` as a success for an operation that failed. Do not include sensitive values in errors.

5.3 Domain/service separation

Route handlers should do only boundary work: parse request, authenticate user, validate input, call a service, map results to HTTP, and emit appropriate telemetry. Business logic belongs in services. Data access belongs in repositories/model functions. Money calculations should be reusable pure functions with unit tests.

5.4 Authorization convention

- Every protected request obtains the authenticated user ID from the server session/token verification process.
- Every record query or mutation must include an ownership constraint, e.g. `{ _id: recordId, userId: authenticatedUserId }`.
- Never accept `userId` from a normal client request as the authority for access.
- For background work, persist the user/account ownership on the job and confirm it against the source resource.
- Use authorization tests that attempt cross-user reads, updates, deletes, imports, exports, and AI requests.

5.5 Money and currency

- Base currency for the initial India-focused release: INR.
- Store INR amounts as integer paise, or use a database-supported exact decimal type with a clear serialization rule.
- Define whether `amountPaise` is always non-negative and direction is represented by `type`, or whether signed amounts are stored. Prefer non-negative absolute amount + explicit transaction type/debit-credit semantics.
- Do not mix signed and unsigned conventions across imports, manual entry, analytics, and forecasts.
- Round only at display boundaries according to documented rules.
- Validate currency codes; multi-currency support is a later feature unless explicitly included.
- Add overflow and boundary-value tests.

5.6 Dates and periods

- Store instants in UTC when a precise instant exists; store transaction calendar dates as a date-only value or a documented UTC-midnight representation without timezone drift.
- Use `Asia/Kolkata` for default user-facing month/day boundaries for the India-first product unless the user selects another timezone.
- Do not parse ambiguous date strings without a known input format.
- Define inclusive/exclusive period boundaries in analytics tests.
- Imported transaction dates often have no time-of-day; do not invent a time.

5.7 Accessibility and UX

- Use semantic HTML and keyboard-accessible controls.
- Provide labels, validation messages, focus handling, and visible focus states.
- Charts must have a textual summary or accessible table alternative.

- All screens must support loading, empty, error, partial data, and success states.
- Destructive actions require clear confirmation or a recoverable pattern.
- Do not rely on color alone for debit/credit, budget status, or errors.

---

6. Data Model and Database Requirements

Use the exact naming convention chosen by the repository. The schemas below are logical requirements; map them to Mongoose models or repository schemas consistently.

6.1 `users`

Field	Type / rule	Purpose
`_id`	ObjectId or chosen stable ID	Primary key
`emailNormalized`	unique, normalized string	Sign-in/recovery lookup
`displayName`	bounded string	Display name
`timezone`	validated IANA timezone, default `Asia/Kolkata`	Period and notification behavior
`currency`	ISO currency code, default `INR`	Display preference
`createdAt`, `updatedAt`	timestamps	Lifecycle
`status`	enum	Active, pending verification, disabled, deletion pending
`preferences`	bounded subdocument	Non-sensitive UI/notification preferences

Authentication provider fields should follow the selected library's documented storage design. Do not store raw passwords if a managed identity provider handles credentials. If password authentication is implemented, use a reputable password hash and do not invent a custom cryptographic scheme.

6.2 `transactions`

Required logical fields:

- `_id`
- `userId` — required owner reference

- `type` — `income` or `expense` (transfers may be added as a distinct type only with defined analytics behavior)
- `amountPaise` — positive integer greater than zero
- `currency` — initially `INR`
- `transactionDate` — normalized calendar date
- `description` — length-limited plain text
- `merchantNormalized` — optional normalized merchant label
- `categoryId` — optional category reference
- `paymentSourceId` or `accountId` — optional owned reference
- `source` — `manual`, `csv_import`, `pdf_import`, or `authorized_provider`
- `importId` — nullable import provenance
- `externalReferenceHash` — optional keyed/hash-safe deduplication value where appropriate
- `categorization` — source, confidence, model/rule version, and review status
- `notes` — optional bounded user text
- `createdAt`, `updatedAt`

Never store complete raw statement rows in the transaction document unless there is a documented reason, bounded size, and retention policy. Preserve import provenance without retaining unnecessary sensitive text.

6.3 `categories`

Fields: `_id`, `userId` or system/category scope, `name`, normalized name, `kind` (income/expense/both), optional icon/color token, `isSystem`, `isArchived`, timestamps. Enforce unique normalized category names per user when appropriate. Reject category IDs that do not belong to the user or valid system scope.

6.4 `accounts / paymentSources`

Represent a user-visible label such as cash, savings account, debit card, or UPI wallet. Fields: `_id`, `userId`, `label`, `type`, optional masked display identifier, `institutionLabel`, `currency`, `status`, timestamps. Do not store banking login credentials, card security codes, PINs, or OTPs. Any account identifier should be minimized and masked.

6.5 `imports`

Fields: `_id`, `userId`, `sourceType`, `originalFileName` (sanitized), `storageKey` (private), `fileMime`, `fileSizeBytes`, `contentHash`, `parserName`, `parserVersion`, `status`, `startedAt`, `completedAt`, `rowCount`, `validCount`, `invalidCount`, `duplicateCount`, `insertedCount`, safe `errorSummary`, and timestamps.

Possible statuses: `uploaded`, `queued`, `processing`, `preview_ready`, `committing`, `completed`, `completed_with_warnings`, `failed`, `cancelled`, `expired`.

Do not store the file's public URL. The storage key must not be guessable and is never returned as an unrestricted external URL.

6.6 `importRows` (optional staging collection)

For preview workflows, keep normalized candidate rows in a staging collection with fields for `importId`, `userId`, row number, normalized candidate transaction fields, validation results, fingerprint, user corrections, and commit status. Apply a short TTL/cleanup policy after completion or expiration. Ensure staging data cannot be queried by another user.

6.7 `budgets`

Fields: `_id`, `userId`, `categoryId` or category scope, `periodType` (`monthly` initially), `periodStart`, `limitPaise`, `currency`, threshold preferences, enabled state, timestamps. Define whether categories can have multiple overlapping budgets; preferably reject conflicting active budgets for the same category/period unless overlap is a deliberate feature.

6.8 `recurringRules`

Fields: `_id`, `userId`, normalized merchant/description key, amount range or expected amount, cadence, nextExpectedDate, linked transaction IDs or match fingerprints, confidence, status (`suggested`, `confirmed`, `muted`, `cancelled`), user confirmation time, timestamps. A detected recurring pattern remains a suggestion until the user confirms it.

6.9 `notifications`

Fields: `_id`, `userId`, `type`, `title`, bounded body, safe navigation target, deduplication key, delivery channels/status, `scheduledAt`, `sentAt`, `readAt`, failure metadata, timestamps. Never store full financial statements in notification payloads. Notifications should be idempotent for a given trigger and period.

6.10 `aiConversations` / `aiMessages` (or a single bounded structure)

Fields must support `userId`, conversation ID, role, bounded message text, safe tool call/result metadata, model/provider identifier, token/cost metadata, safety outcome, and timestamps. Define a message retention/deletion policy. Avoid duplicating transaction details in long-term chat logs where short-lived summaries or references suffice. Never persist system prompts containing secrets.

6.11 `auditEvents`

Fields: `_id`, actor/user ID, action enum, target type and safe target ID, outcome, request ID, timestamp, limited non-sensitive metadata. Audit authentication

changes, account export/deletion, statement upload/commit, category/budget changes, and administrator actions if any. Do not log passwords, tokens, statement rows, message content, or raw financial values unnecessarily.

6.12 `backgroundJobs` / queue payloads

Use a durable job system appropriate to deployment. Job payloads should contain IDs and minimal options, not raw statement files, credentials, or complete financial histories. Record attempts, timestamps, safe error codes, and correlation ID. Set maximum retries and a dead-letter/review path.

6.13 Required indexes

At minimum, review and add indexes matching real queries:

- `users.emailNormalized` unique.
- `transactions: { userId: 1, transactionDate: -1, _id: -1 }`.
- `transactions: { userId: 1, categoryId: 1, transactionDate: -1 }` where category reports need it.
- `transactions: { userId: 1, importId: 1 }` where import traceability is needed.
- Unique/indexed import fingerprints scoped by user and account/source, where the chosen dedupe model supports safe uniqueness.
- `imports: { userId: 1, createdAt: -1 }`.
- `budgets: { userId: 1, periodStart: 1, categoryId: 1 }` with a suitable uniqueness strategy.
- `notifications: { userId: 1, createdAt: -1 }` and unique deduplication key scope.
- TTL index for temporary staging data only where retention behavior is understood and validated.

Index design must be validated with representative explain plans. Do not add a broad index to every field by default.

6.14 Database integrity rules

- Validate critical fields both at the API/service layer and, where possible, at database schema level.
- Use transactions/sessions for multi-document commits when supported by the deployed MongoDB topology; otherwise implement a safe state machine, uniqueness constraints, and recovery process.
- Every monetary record must have a consistent direction and currency convention.
- Avoid cascading deletion without an explicit data-retention and recovery decision.
- Include schema versioning or a documented migration strategy for evolving records.

7. API Technical Contract

All paths below are illustrative version-one REST endpoints. Adapt route naming to the existing app. A breaking public API should be versioned rather than silently changed.

7.1 Auth and profile

Method	Endpoint	Purpose	Important rules
`GET`	`/api/health`	Safe application health	Do not expose environment, hostnames, or secrets.
`GET`	`/api/me`	Current user's profile	Session required.
`PATCH`	`/api/me`	Update allowed profile/preferences	Allowlist editable fields.
`POST`	`/api/account/export`	Start data export	Auth, rate limit, private artifact, expiration.
`POST`	`/api/account/delete`	Request account deletion	Re-authentication/confirmation as appropriate.

Registration/login/session routes may be owned by the chosen authentication library; do not recreate library routes without need.

7.2 Transactions

Method	Endpoint	Purpose
`GET`	`/api/transactions`	List, filter, sort, and paginate owned transactions
`POST`	`/api/transactions`	Create manual transaction
`GET`	`/api/transactions/:id`	Read one owned transaction
`PATCH`	`/api/transactions/:id`	Update allowlisted fields on an owned transaction

Method	Endpoint	Purpose
DELETE	/api/transactions/:id	Delete an owned transaction with defined summary recalculation behavior

Requirements: - Filter fields: date range, type, category, source, account/payment source, search query. - Validate page size and use stable ordering (date then ID) to prevent unstable paging. - Never accept raw MongoDB filter JSON from clients. - Validate any referenced category/account against the authenticated user. - Enforce maximum search length and rate-limit expensive queries.

7.3 Categories and payment sources

Method	Endpoint	Purpose
GET/POST	/api/categories	List/create owned categories
PATCH/DELETE	/api/categories/:id	Update/archive/delete owned category, with transaction migration behavior defined
GET/POST	/api/payment-sources	List/create labels such as cash or UPI
PATCH/DELETE	/api/payment-sources/:id	Update/archive source

7.4 Statement imports

Method	Endpoint	Purpose
POST	/api/imports	Create upload/import job or request a short-lived upload authorization
GET	/api/imports	List current user's imports
GET	/api/imports/:id	Read import status and summary
POST	/api/imports/:id/preview	Trigger/refresh parsing and validation preview
PATCH	/api/imports/:id/rows	Correct eligible staging rows or column mapping

Method	Endpoint	Purpose
POST	/api/imports/:id/commit	Commit validated rows idempotently
POST	/api/imports/:id/retry	Retry a recoverable failed job
DELETE	/api/imports/:id	Cancel/remove an eligible import and its temporary artifact

Import requirements: - Require an authenticated owner for all import routes. - Verify MIME type and file contents, not filename extension alone. - Enforce size, row-count, processing-time, and decompression limits. - Protect against malformed PDFs, archive/decompression bombs if archives are ever supported, formula injection in exported CSVs, and parser resource exhaustion. - For large files, process asynchronously and return a job identifier/status. - Commit with dedupe constraints and a clear partial-failure policy. - File names are untrusted display text; sanitize them before rendering or logging.

7.5 Analytics and budgets

Method	Endpoint	Purpose
GET	/api/analytics/summary	Income/expense/net values for a period
GET	/api/analytics/trends	Daily/monthly trends
GET	/api/analytics/categories	Category spend breakdown
GET/POST	/api/budgets	List/create budgets
GET/PATCH/DELETE	/api/budgets/:id	Read/update/delete one owned budget
GET	/api/recurring	List recurring candidates/confirmed rules
POST	/api/recurring/:id/confirm	Confirm a candidate
POST	/api/recurring/:id/mute	Mute a recurring candidate

Analytics must use one shared, tested definition for included transaction types, date periods, refunds/adjustments, and transfers. Avoid computing authoritative totals differently in the browser and server.

7.6 AI endpoints

Method	Endpoint	Purpose
`POST`	`/api/ai/categorize`	Categorize one or a bounded batch of transactions
`POST`	`/api/ai/chat`	Ask a personal-finance question using authorized, grounded context
`GET`	`/api/ai/usage`	Optional usage/quota view for the current user

AI requirements: - User identity comes from server auth, not request body. - The client cannot provide arbitrary prompts that overwrite system policy or arbitrary database queries. - AI output is parsed against a strict schema; invalid output is rejected or retried within a limit. - Any tool call must be mapped to a small allowlist with validated arguments and explicit user scope. - Apply per-user rate limits, timeouts, token/output limits, and provider spending controls. - Handle provider failure with a clear fallback and no fabricated answer. - Show when an answer is estimated, inferred, or based on incomplete/imported data.

7.7 Status and response semantics

- `400` malformed request, `401` unauthenticated, `403` authenticated but disallowed, `404` missing/inaccessible resource (avoid revealing cross-user existence), `409` conflict/duplicate/state mismatch, `413` payload too large, `422` semantic validation failure where used consistently, `429` rate limit, `500` unexpected server error, `503` temporary dependency unavailable.
- Choose one consistent policy for `403` versus `404` on cross-user identifiers; do not reveal whether another user's record exists.
- Define idempotency keys for expensive create/commit operations where retries are possible.

---

8. Phase-by-Phase Implementation Plan

Each phase must be reviewed against the repository before code changes. Phases are sequential by default; a later phase cannot be considered complete if an earlier dependency is unstable.

Phase 0 — Repository Audit and Technical Baseline

Goal: Understand current state and avoid unnecessary rewrites.

Technical work - Inspect the repository tree, package scripts, framework versions, existing routes, authentication, schema/model layer, and environment

configuration. - Run the existing app, tests, lint, type-check, and production build where possible. - Record current functionality, failing checks, security risks, technical debt, and feature flags. - Compare implementation against this TRD and the PRD. - Produce a gap report with a recommended next checkpoint.

Deliverables: baseline report, verified run commands, risk list, implementation plan, and current test/build results.

Exit criteria: no source rewrite begins before the audit is documented; existing failures are identified and classified.

Phase 1 — Project Setup and Developer Experience

Goal: Establish repeatable development and deployment foundations.

Technical work - Configure Next.js App Router, strict TypeScript, Tailwind CSS, reusable UI primitives, linting, formatting, and scripts. - Add server-only environment validation and `.env.example`. - Configure database connection reuse for serverless/runtime behavior appropriate to hosting. - Add a safe health route and a consistent error boundary. - Add CI checks for lint, type-check, tests, and production build. - Document local setup, database setup, seed/demo data, and troubleshooting.

Exit criteria: local setup is repeatable from a clean checkout; missing configuration fails safely; no secret is in Git or client bundles; CI runs the required checks.

Phase 2 — Authentication, Authorization, and Account Lifecycle

Goal: Prevent access to private finance data by unauthorized users.

Technical work - Configure the selected authentication provider/library. - Implement registration, login, logout, recovery, verification where used, and session revocation behavior. - Configure secure cookies/session expiry and origin/CSRF protection according to the selected auth model. - Add an authenticated user context helper for route handlers and services. - Centralize authorization checks and safe error responses. - Add audit events for critical account actions. - Implement account status and deletion/export state models, even if export/deletion is delivered in a later phase.

Exit criteria: unauthenticated access is blocked; cross-user object reference tests fail safely; sensitive tokens are not logged; recovery flows expire safely; security checks pass.

Phase 3 — Transaction, Category, and Payment-Source Core

Goal: Create a reliable source of truth for personal finance records.

Technical work - Implement database models/schemas and migrations/versioning policy. - Implement manual create/read/update/delete and category/payment-source management. - Add reusable money/date validation and calculation helpers. - Add search, filter, sort, pagination, and empty/error/loading states. - Use server-side totals as the single source of truth. - Build synthetic seed fixtures and unit/integration tests.

Exit criteria: CRUD works for an owner only; money totals match fixture calculations; references are validated; pagination/filtering is stable; no floating-point authoritative totals exist.

Phase 4 — Statement Upload, Parsing, Preview, and Commit

Goal: Import data safely without duplicates or silent corruption.

Technical work - Implement a private upload process, import metadata, state machine, and temporary artifact cleanup. - Support CSV using explicit delimiters/encoding/date/amount parsing and column mapping. - Add parser adapters for individually tested SBI PDF statement layouts; do not claim universal PDF support. - Normalize debit/credit, date, description, amount, optional balance/reference into candidate transactions. - Add import preview with per-row errors, user corrections, and explicit confirmation. - Implement account/source-scoped deduplication, stable external references where available, and a safe fallback fingerprint. - Add queue-backed processing for files too large for a request lifecycle. - Commit rows idempotently, with transaction/session or compensating-recovery strategy. - Record parser version, counts, warnings, and safe error reasons.

Exit criteria: representative files pass verified parser fixtures; repeated imports create no unintended duplicates; failed/retried commits are safe; temporary files expire; invalid rows are visible and not silently dropped.

Phase 5 — Dashboard, Analytics, and Reports

Goal: Give users accurate, useful summaries of imported and manually entered records.

Technical work - Implement summary, trend, category, and recent-transaction queries. - Establish common calculation semantics and date boundaries. - Add period filters and disclose the scope of available data. - Use indexed queries, pagination, and bounded aggregation ranges. - Add chart accessibility alternatives and no-data states. - Test analytics with deterministic fixtures including refunds, same-day duplicates, missing categories, and month boundaries.

Exit criteria: dashboard values match independent expected results; filters are consistent across cards/charts/tables; performance meets defined targets on representative datasets; stale/partial data is clearly indicated.

Phase 6 — AI-Assisted Categorization

Goal: Reduce manual categorization while keeping users in control.

Technical work - Start with a deterministic rules layer for obvious merchant/description matches. - Add an AI provider adapter with bounded input, output schema, timeout, retry, and token/cost limits. - Pass only fields needed for categorization; redact irrelevant account identifiers and notes. - Return candidate category, confidence band, short rationale, and model/rule version. - Validate category IDs against the user's allowed category set. - Keep low-confidence outputs as suggestions; allow confirm, edit, and reject. - Store provenance and feedback to improve rules without silently retraining on private data. - Provide a non-AI manual fallback during provider outage.

Exit criteria: schema-invalid model output is rejected; category references cannot cross users; confidence handling is tested; no private full statement is sent unnecessarily; user corrections override suggestions.

Phase 7 — Grounded AI Finance Assistant

Goal: Answer finance questions using verified application data rather than hallucinated figures.

Technical work - Create an AI gateway separate from UI and route code. - Implement allowlisted server tools such as `get_period_summary`, `get_category_spend`, `get_budget_status`, and `get_recurring_candidates`. - Every tool must derive user identity from the trusted session, validate date ranges, limit result size, and return only required aggregates. - Keep authoritative aggregation inside deterministic service functions; the model may summarize the returned results but must not determine arithmetic. - Constrain prompts, tool schemas, context length, message length, and conversation history size. - Defend against prompt injection from user text, merchant strings, imported descriptions, and retrieved records. - Validate that responses do not invent transactions, balances, dates, or guaranteed outcomes. - Add safety copy for financial guidance and direct users to qualified professionals for regulated or individualized investment/tax/legal decisions. - Implement user-visible AI limitations, logging minimization, retention controls, and usage caps.

Exit criteria: answers cite or describe the period/data used; arithmetic matches backend values; no arbitrary query tool exists; prompt injection tests pass; outage produces a clear fallback instead of an invented answer.

Phase 8 — Budgets, Thresholds, and Notifications

Goal: Help users set limits and receive predictable, non-duplicated alerts.

Technical work - Implement budget CRUD, period semantics, amount validation, and category association. - Define whether spending is based on transaction date or import/posted date; use transaction date for user-facing budget calculations unless the product explicitly says otherwise. - Calculate thresholds on the server and trigger notifications through a durable scheduled job or event workflow. - Deduplicate alerts by budget, threshold, and period. - Support in-app notices first; add email only after provider configuration and verification. - Add notification preferences, mute controls, timezone-aware delivery, retry limits, and failure visibility.

Exit criteria: spending threshold results match fixtures; alerts do not duplicate on job retries; disabled/muted preferences are respected; provider outages do not block transaction entry.

Phase 9 — Recurring Payment Detection

Goal: Identify likely recurring income and expenses transparently.

Technical work - Build candidate generation using merchant similarity, date cadence, and amount tolerance. - Use explainable scoring rules before optionally adding ML. - Keep candidate records separate from confirmed recurring rules. - Let users confirm, edit expected dates/amounts, mute, or dismiss candidates. - Track false positives and edge cases such as yearly bills, variable utilities, split payments, and subscriptions with price changes. - Generate reminders only for confirmed rules or clearly labeled suggestions.

Exit criteria: detection is tested against synthetic recurring/non-recurring fixtures; candidates never become confirmed silently; users can correct or dismiss records; reminder jobs are idempotent.

Phase 10 — Cash-Flow Forecasting

Goal: Offer an estimate of near-term cash flow with transparent assumptions.

Technical work - Define minimum history and data-quality thresholds before enabling forecasts. - Use confirmed recurring items and historical trends with a documented baseline model. - Show forecast horizon, historical lookback, included/excluded categories, and uncertainty/range where meaningful. - Separate actual transactions from forecast rows in data models and UI. - Add evaluation fixtures for irregular income, missing months, recent account changes, and limited imported history. - Do not treat uncertain recurring candidates or stale partial statement balances as facts.

Exit criteria: forecast is labeled estimate; assumptions are visible; insufficient

data disables or qualifies results; model is evaluated against a held-out or synthetic scenario set; no guarantee language is used.

Phase 11 — Export, Deletion, Retention, and Privacy Controls

Goal: Give users usable control over their data lifecycle.

Technical work - Export transactions, categories, budgets, and appropriate profile data in documented machine-readable formats. - Generate exports asynchronously when needed, store them privately, expire links, and restrict them to the owner. - Implement account deletion as a state machine with confirmation, active-session invalidation, job cancellation, storage cleanup, and database deletion/anonymization according to policy. - Define retention for raw statement files, staging rows, logs, AI conversations, failed jobs, exports, and backups. - Document what cannot be immediately removed from backups and the expiration plan, consistent with the adopted policy and applicable law. - Test partial deletion retries and orphan-file cleanup.

Exit criteria: exports contain only the requesting user's data; temporary links expire; deletion is auditable and retryable; retention jobs are monitored; policy text matches actual system behavior.

Phase 12 — Production Hardening and Release Readiness

Goal: Establish operational safety, resilience, and maintainability.

Technical work - Complete dependency scanning, secret scanning, SAST, and suitable dynamic security testing. - Add rate limiting, upload limits, CSRF protections where relevant, safe headers, and content security policy appropriate to the app. - Configure production secrets, HTTPS, staging, backups, restore tests, alerting, and rollback procedure. - Add request/job correlation IDs, safe structured logs, latency/error metrics, queue-depth tracking, provider quota monitoring, and uptime checks. - Add load tests for core APIs, import processing, analytics, and AI limits. - Validate accessibility, responsive layout, browser compatibility, and degraded-mode UI. - Create incident-response steps for data exposure, provider outage, failed import, accidental deletion, and credential rotation. - Verify legal/privacy terms and vendor agreements with the appropriate qualified reviewers before launch.

Exit criteria: production build and automated checks pass; backup restoration is tested; no unresolved critical/high security issue remains without an approved mitigation; monitoring and rollback are operational; release checklist is signed off.

Phase 13 — Optional Authorized Bank-Data Integration

Goal: Explore a legitimate, consent-based bank data connection only after core release and feasibility review.

Technical work - Research and verify current eligibility, regulatory requirements, contracts, supported institutions, consent flow, data scope, refresh behavior, revocation, and provider security controls. - Use an eligible authorized provider or applicable consent-based ecosystem where legally and contractually permitted. - Implement integration behind a feature flag and provider adapter; keep upload/manual flows available. - Store provider connection identifiers and consent/expiry status, not bank login passwords or OTPs. - Validate webhook authenticity and idempotency if webhooks are supported. - Show connection state, last successful sync, stale-data warnings, disconnect, and consent revocation behavior. - Implement reconciliation/deduplication against imported data without accidentally double-counting transactions.

Exit criteria: no integration is enabled until eligibility and provider arrangements are verified; consent/revocation and security tests pass; sync failures are visible; manual/upload alternatives continue to work.

---

9. Statement Import Engineering Detail

9.1 Import state machine

```
uploaded -> queued -> processing -> preview_ready -> committing -> completed
                              |              |              |
                              +-> failed     +-> cancelled  +-> completed_with_warnings
```

Allowed state transitions must be enforced server-side. A client cannot directly set an import to `completed`. Retries should create a new attempt or return a known state without duplicating committed records.

9.2 Parser adapter contract

Each parser should implement a common contract similar to:

```ts
type ParseContext = {
  userId: string;
  importId: string;
  parserVersion: string;
  timezone: string;
};

type ParsedCandidate = {
  rowNumber: number;
  transactionDate: string;
```

```
  description: string;
  amountPaise: number;
  direction: "debit" | "credit";
  currency: "INR";
  externalReference?: string;
  balanceAfterPaise?: number;
  warnings: string[];
};

interface StatementParser {
  canParse(metadata: { mimeType: string; headerSample?: string }): boolean;
  parse(input: Uint8Array, context: ParseContext): Promise<{
    candidates: ParsedCandidate[];
    rowErrors: Array<{ rowNumber: number; code: string; message: string }>;
    warnings: string[];
  }>;
}
```

This is illustrative TypeScript, not a complete parser implementation. Keep parser errors safe; avoid dumping full row contents into logs.

9.3 CSV safety

- Explicitly handle UTF-8 and known encoding options where practical.
- Support quoted fields and escaped delimiters using a maintained CSV parser.
- Limit columns, rows, field length, and total processing time.
- Detect headers instead of relying on fixed column positions alone.
- Make date and amount formats explicit and test common negative/credit/debit notations.
- Do not execute CSV cells as formulas. Escape leading formula characters when exporting data to CSV viewed in spreadsheet applications.

9.4 PDF safety

- Accept only the documented PDF formats and parser route.
- Treat PDFs as untrusted input and run parsing in a bounded worker/process where practical.
- Limit file size, page count, extraction time, and memory.
- Do not execute embedded scripts or open external links from uploaded content.
- Use text extraction when layout is supported; OCR should be a distinct, opt-in fallback only if privacy, cost, latency, and accuracy are understood.
- Compare parser output against manually reviewed sample statements and include a versioned fixture set without real customer data.

9.5 Reconciliation and duplicate detection

- Prefer bank-provided transaction reference/UTR/unique IDs only when stable and available.
- Fallback fingerprints may include account/source scope, normalized date, amount, direction, normalized description, and reference hash.
- Use a keyed hash or suitable non-reversible representation where sensitive values could otherwise be exposed in fingerprints.
- Present ambiguous duplicates to the user; do not silently remove legitimate same-day identical purchases.
- Define behavior for refunds, reversals, and balance-forward rows.
- Verify that statement opening/closing balances are metadata, not transactions, unless the format clearly contains a real transaction row.

---

10. Analytics Definitions

Write and test canonical calculation functions. Do not reimplement business rules in React components.

10.1 Basic formulas

- **Total income:** sum of eligible income transaction amounts in the requested period.
- **Total expense:** sum of eligible expense transaction amounts in the requested period, excluding transfer rows unless product rules state otherwise.
- **Net cash flow:** total income minus total expense for the same period and same data scope.
- **Budget used:** eligible spending for the budget category and period divided by budget limit, with explicit behavior for a zero/missing limit.
- **Category share:** category expense divided by total eligible expense for the same period; display as unavailable when the denominator is zero.

10.2 Policy decisions to document and test

- How refunds and reversals affect expense totals and category trends.
- Whether uncategorized transactions appear in an `Uncategorized` bucket.
- Whether transfers between a user's own accounts are excluded from income/expense totals.
- Which transaction date drives period inclusion.
- Whether deleted transactions are removed from totals immediately or soft-deleted.
- How imports overlapping a prior date range affect summaries.
- How missing historical records affect comparisons and forecasts.

10.3 Performance approach

- Begin with indexed aggregations and bounded date ranges.
- Add cached/precomputed summaries only after profiling shows a need.
- If caching is introduced, document invalidation after create/update/delete/import commit and never serve one user's cached data to another user.
- Set query timeout and maximum range limits for expensive analytics routes.

---

11. AI Architecture, Safety, and Reliability

11.1 Provider adapter

Define a server-only abstraction such as:

```
interface AIProvider {
  generateStructured<T>(input: {
    purpose: "categorization" | "finance_assistant";
    systemPolicyVersion: string;
    userContent: string;
    schema: unknown;
    maxOutputTokens: number;
    timeoutMs: number;
  }): Promise<{ output: T; model: string; usage?: { inputTokens?: number; outputTokens?: nu
}
```

Use the chosen provider's supported structured-output mechanism where available. Validate parsed outputs even when the provider claims schema conformance.

11.2 Data minimization

- Send only necessary transaction fields; do not send entire bank statements by default.
- Redact full account numbers, reference identifiers, email/phone, and unrelated notes unless explicitly necessary and justified.
- Bound the number of transactions and historical period sent to a request.
- Do not include data from another user, global search, or unrestricted database dumps in context.
- Define whether the selected AI provider may retain requests and verify its current contractual/privacy options before production.

11.3 Allowed tools for the assistant

Examples of narrow tool functions: - `get_period_summary(startDate, endDate)` - `get_category_spend(startDate, endDate, categoryId?)` - `get_budget_status(periodStart)` - `get_recurring_candidates(limit)` -

`search_transactions(filters, limit)` with strict allowlisted filters and capped results

Every tool must use authenticated server-side user scope; validate dates/IDs; limit results; return structured data; and reject arbitrary database operators, code, shell commands, or URLs. Do not expose a generic `runMongoQuery` tool to the model.

11.4 Prompt-injection defenses

- Treat transaction descriptions, merchant names, uploaded text, and user-supplied content as data, not system instructions.
- Separate trusted developer policy from untrusted retrieved content.
- Use a narrow tool allowlist and authorization checks outside the model.
- Validate outputs and verify any factual numeric claim against tool results.
- Add tests containing malicious text in merchant descriptions and imported statement rows.

11.5 Honest responses

The assistant must state when data is incomplete, imported only through a given date, absent, or insufficient for a reliable conclusion. It must not claim that it checked a live bank account unless an authorized live provider response supports that statement. Never produce guaranteed-return promises or individualized high-risk investment instructions.

11.6 Resilience and cost controls

- Apply per-user and global rate limits.
- Set request timeout, token/output cap, maximum conversation history, and maximum batch size.
- Retry only transient failures with bounded exponential backoff and jitter.
- Do not retry validation/safety failures indefinitely.
- Track cost/usage without logging prompts or transaction details.
- Provide fallback to deterministic rules or a helpful unavailable message during provider failure.

---

12. Security Requirements

12.1 Authentication and sessions

- Use an established provider/library and supported security configuration.
- Use secure, HttpOnly, SameSite cookies where cookie sessions are used; set `Secure` in HTTPS production.
- Rotate/revoke sessions according to password changes, account recovery, and account deletion policy.

- Apply MFA if a suitable supported provider and the product's threat model justify it; do not claim MFA is enabled if it is not.
- Rate-limit login, recovery, verification, export, import, and AI endpoints.

12.2 Input and application security

- Validate all inputs on the server; escape/encode outputs in their target context.
- Prevent injection by using typed driver/model APIs and never concatenating untrusted input into database operators or queries.
- Add CSRF protections where the auth/session architecture requires them.
- Configure appropriate security headers and a restrictive Content Security Policy compatible with the deployed app.
- Prevent open redirects and unsafe URL handling.
- Protect against stored/reflected XSS, SSRF, path traversal, unsafe file parsing, mass assignment, and insecure direct object references.
- Do not trust MIME types or client-provided content lengths.

12.3 Financial-data protection

- TLS for transit and managed encryption at rest for database and storage.
- Least-privilege database and storage credentials.
- Private storage buckets, limited access policies, and short-lived upload/download access when required.
- Mask account identifiers in the UI and logs.
- Keep sensitive fields out of analytics/telemetry by default.
- Define and test backup retention, access, and restoration.
- Document incident response and credential-rotation procedures.

12.4 Authorization tests

Automated tests must verify that User A cannot access User B's: - transactions, categories, accounts/payment sources, budgets, imports, staged rows, recurring rules, notifications, AI conversations, exports, or deletion requests. - records by modifying URL IDs, query parameters, payload IDs, or job IDs. - AI-derived summaries through another user's request or cached result.

12.5 Threat review checklist

Before release, review at least: account takeover, broken access control, session theft, leaked environment variables, unsafe file upload/parser, malicious CSV/PDF content, prompt injection, over-permissive AI tools, duplicate import race conditions, expensive query abuse, stored XSS, export URL leakage, insecure backups, log leakage, and provider outage/cost spikes.

---

13. Privacy, Compliance, and Responsible Use

- Publish privacy and retention statements that accurately reflect implementation.
- Collect only data needed for stated features.
- Obtain clear user consent for imports, AI processing, emails, and optional integrations where appropriate.
- Verify current India-specific legal and regulatory requirements with qualified counsel before launch. This document is not legal advice and does not certify compliance.
- Do not imply that a generic finance assistant is a licensed financial advisor.
- Provide user controls for deleting uploaded files, AI history, account data, and optional connections according to the retention policy.
- Ensure email, AI, file storage, analytics, and monitoring vendors are reviewed for data handling and access controls.
- Do not use customer financial records for model training unless there is a clearly disclosed, lawful, opt-in process and approved provider terms.
- Use synthetic or anonymized fixtures in engineering and support workflows.

---

14. Non-Functional Requirements (NFRs)

These targets are initial engineering goals and should be confirmed or adjusted after a representative staging load test.

Area	Initial target / requirement
Reliability	Core transaction CRUD should degrade gracefully when optional AI/email providers are down.
API performance	Typical authenticated CRUD and summary routes should aim for p95 under 500 ms on representative data, excluding cold starts/external dependencies; measure rather than assume.
UI performance	Avoid unnecessary client-side JavaScript; prioritize interactive dashboard controls and loading states. Use production bundle analysis.
Import responsiveness	Create an import job quickly; parse large files asynchronously with visible progress/status.

Area	Initial target / requirement
Scalability	Paginate collections and bound date range, upload size, rows per import, AI context, and query cost.
Security	No known unresolved critical/high risk at release without a formally accepted mitigation.
Accessibility	Target WCAG 2.2 AA practices where feasible; verify keyboard, focus, labels, contrast, and screen-reader behavior.
Browser support	Test current stable Chrome, Edge, Firefox, and Safari; document minimum supported versions.
Observability	Correlated structured logs, error tracking, queue/job metrics, latency/error monitoring, uptime checks.
Recoverability	Backups configured and restoration tested before production release.
Maintainability	Strict types, shared domain services, meaningful tests, architecture notes, and documented setup.
Data retention	Temporary uploads/staging/export artifacts expire according to a defined policy.
Cost control	AI, file processing, email, and analytics costs have quotas/alerts or explicit budget limits.

---

15. Testing Strategy

15.1 Unit tests

- Money conversions, sums, formatting, and boundary cases.
- Date parsing, month boundaries, timezone behavior, and ranges.
- Categorization rules and confidence thresholds.
- CSV and supported PDF parser fixtures.
- Deduplication fingerprints and ambiguous duplicate behavior.
- Budget thresholds, recurring cadence, notification dedupe, and forecast logic.
- AI response schemas and error mapping.

15.2 Integration tests

- Database schema and indexes.
- Repository ownership filters.
- API input validation and status codes.
- Authentication/session integration.
- Import state transitions, idempotency, retries, and partial failures.
- Queue job authorization, retries, and dead-letter behavior.
- Account export/deletion and storage cleanup.
- AI adapter timeouts, quota handling, and provider errors.

15.3 End-to-end tests

1. Register/sign in and reach a private dashboard.
2. Create, edit, filter, and delete a transaction.
3. Create a category and assign a transaction.
4. Import a valid CSV, inspect preview, correct a row, and commit.
5. Re-upload the same statement and verify deduplication behavior.
6. View analytics across month boundaries.
7. Create a budget and trigger a threshold test with deterministic fixtures.
8. Request an AI summary and verify figures correspond to backend values.
9. Attempt to access another user's transaction/import/export and verify denial.
10. Export account data and verify it is private and expires.
11. Delete an account in staging and verify session revocation/cleanup behavior.
12. Simulate AI provider and email outages and verify core transaction operations still work.

15.4 Security and resilience tests

- Invalid/oversized uploads, malformed PDFs, extreme CSV rows, and parser timeouts.
- Rate limit behavior and abuse controls.
- CSRF/XSS/access-control regression tests.
- Prompt injection inside user messages and transaction descriptions.
- Duplicate import requests racing concurrently.
- Worker crash and retry behavior.
- Database outage and recovery.
- Backup restore drill.

15.5 Release gates

A phase cannot be marked complete until: - Tests for the phase pass. - Typecheck, lint, and production build pass. - Known failure modes and unresolved issues are recorded. - Authorization and sensitive-data implications are reviewed.

- Acceptance criteria are checked by a person or an explicitly documented automated test.

---

16. Deployment and Operations

16.1 Environments

Maintain at least local development, staging, and production environments. Use separate database instances/credentials, AI keys, storage buckets, email settings, and session secrets. Never run destructive tests against production data.

16.2 CI/CD pipeline

Recommended pipeline: 1. Install from the lockfile. 2. Lint and format check. 3. Type-check. 4. Unit tests. 5. Integration tests against an isolated test database. 6. Dependency/secret scanning. 7. Production build. 8. Deploy to staging. 9. Run smoke tests and migration/index checks. 10. Promote to production with a rollback plan.

Do not run destructive database migration logic during an unreviewed request. Treat migrations and index changes as versioned deployment artifacts.

16.3 Database and storage operations

- Use separate least-privilege credentials for app and administrative operations.
- Define backup frequency and retention based on the recovery point objective (RPO) and recovery time objective (RTO) selected by the owner.
- Test restoration regularly, not only backup creation.
- Monitor connection pool usage, slow queries, storage growth, failed jobs, and import error rates.
- Add retention cleanup and orphan-artifact cleanup jobs with observable success/failure metrics.

16.4 Logging and monitoring

Log request ID, route/module, status, latency, user ID or pseudonymous ID where operationally necessary, error code, and job ID. Do not log passwords, sessions, authorization headers, full prompts, statement rows, full account numbers, or raw provider secrets. Apply log access control and retention limits.

16.5 Incident and rollback readiness

Document procedures for: - leaked secret or compromised account; - unauthorized access or suspected data disclosure; - bad import/parser release; - wrong dashboard calculations; - AI provider outage or unexpected cost; - stuck queue jobs; - accidental deletion or corrupt migration; - notification delivery failure.

Each procedure should define detection, containment, recovery, communication owner, evidence preservation, and post-incident review.

---

17. AI Coding Agent / Developer Execution Protocol

Use the following master instruction at the beginning of each implementation checkpoint:

> You are contributing to PaisaPilot, a personal finance management and insight application. First inspect the repository and current phase status. Do not rewrite working modules or assume external credentials/integrations exist. Implement only the requested phase and its dependencies. Keep all sensitive operations server-side. Derive the user identity from verified authentication, enforce ownership on every resource access, validate every untrusted input with runtime schemas, and use exact monetary arithmetic (integer paise or an exact decimal type). Never invent bank connectivity, transaction data, test results, or deployment success. Do not expose bank passwords, OTPs, PINs, session tokens, secrets, statement contents, or raw AI prompts in logs. AI may use only approved backend tools and must not execute arbitrary database queries. Add or update unit, integration, and relevant end-to-end tests. Run lint, type-check, tests, and production build. Fix regressions caused by the change. At the end, report: files changed, behavior implemented, commands run, actual results, security considerations, known limitations, and the exact next checkpoint. Mark the phase complete only when its exit criteria pass.

17.1 Required end-of-phase report

```
Phase:
Repository baseline:
Implementation summary:
Files changed:
Database/schema changes:
API/UI changes:
Security and authorization review:
Tests run and actual results:
Lint/type-check/build results:
Manual verification performed:
Known limitations or blockers:
Phase acceptance criteria: PASS / PARTIAL / FAIL
Next recommended phase:
```

The agent must not claim tests passed unless commands actually ran and their outcomes are known. If the environment cannot run a check, report it as not

run and explain why.

---

18. Definition of Done (Global)

A feature or phase is complete only when all applicable conditions are met:

- ☐ Requirements and edge cases are documented.
- ☐ Server-side validation and authorization are in place.
- ☐ Database changes and indexes are documented and reviewed.
- ☐ Loading, empty, success, error, and partial-data states exist where appropriate.
- ☐ Money and date behavior is covered by tests.
- ☐ Cross-user access is denied and tested.
- ☐ Privacy, logging, retention, and AI/provider implications are reviewed.
- ☐ Unit/integration/E2E tests appropriate to the change pass.
- ☐ Lint, type-check, and production build pass, or blockers are clearly recorded.
- ☐ Documentation and `.env.example` are updated without secrets.
- ☐ Failure and retry behavior has been tested for background processes.
- ☐ A rollback or recovery path exists for risky data changes.
- ☐ The phase report reflects the true repository state.

---

19. Release Checklist

Product and data correctness

- ☐ Financial calculations reconcile to known test fixtures.
- ☐ Statement parser support is limited to tested formats and communicated clearly.
- ☐ Import retries and duplicate detection are safe.
- ☐ Partial/stale data is clearly labeled.
- ☐ Forecasts and recurring-payment predictions are identified as estimates/suggestions.

Security and privacy

- ☐ All protected endpoints have authorization checks.
- ☐ Cross-user IDOR tests pass.
- ☐ Secrets are server-only and secret scanning is clean.
- ☐ Upload validation, size caps, parser limits, and cleanup jobs are active.
- ☐ AI tool access is allowlisted and output schema-validated.
- ☐ Retention and deletion behavior matches user-facing policy.
- ☐ Legal/privacy reviews required for launch have been completed by qualified reviewers.

Operations

- ☐ Staging smoke tests pass.
- ☐ Backups and a restore drill are complete.
- ☐ Alerts for errors, job failures, and unusual cost/usage are active.
- ☐ Rollback and incident-response instructions are available.
- ☐ Provider outages do not prevent core transaction CRUD.
- ☐ Support/feedback route and a known-issues list are available.

---

20. Open Technical Decisions

Resolve these choices explicitly before the related phase; do not silently assume answers:

1. Authentication library/provider and password/MFA policy.
2. Mongoose versus official MongoDB driver; use one primary access pattern.
3. Background job queue and worker hosting model.
4. Private object storage provider and retention period for original statement files.
5. Exact SBI PDF formats/statement samples to support and how to maintain parser fixtures safely.
6. AI provider, privacy/retention settings, structured-output support, per-user quotas, and budget ceiling.
7. Email provider, sender-domain verification, and notification defaults.
8. Soft deletion versus hard deletion for transactions and categories.
9. Policy for refunds, reversals, transfers, and category budgets.
10. Analytics performance targets and expected transaction volume.
11. Data export formats and account deletion/backups policy.
12. Applicable legal, privacy, vendor, and regulated integration review before production release.

Each decision should be recorded in a lightweight ADR (Architecture Decision Record) containing context, options, decision, trade-offs, and date.

---

21. Suggested Architecture Decision Records

Create concise records for at least:

- ADR-001: Next.js modular monolith and worker boundary.
- ADR-002: MongoDB access pattern and schema validation.
- ADR-003: Authentication and session management.
- ADR-004: INR amount representation and date policy.
- ADR-005: Import state machine and deduplication strategy.
- ADR-006: Temporary statement storage and retention.
- ADR-007: AI provider, data minimization, and tool policy.

- ADR-008: Queue provider, retries, idempotency, and dead-letter handling.
- ADR-009: Account export/deletion and backup retention.
- ADR-010: Future bank-data integration eligibility and provider decision.

---

22. Recommended Implementation Order

```
Phase 0: audit repository
  -> Phase 1: setup + CI
  -> Phase 2: authentication + authorization
  -> Phase 3: transaction/category core
  -> Phase 4: statement import
  -> Phase 5: dashboard + analytics
  -> Phase 6: AI categorization
  -> Phase 7: grounded AI assistant
  -> Phase 8: budgets + notifications
  -> Phase 9: recurring detection
  -> Phase 10: forecasting
  -> Phase 11: export/deletion/retention
  -> Phase 12: production hardening
  -> Phase 13: optional authorized bank integration
```

Some privacy/deletion foundations, audit events, logging, and threat review must start early even if their complete user-facing workflows finish in Phase 11 or 12. Do not postpone basic security until the end.

---

23. Final Technical Direction

Build PaisaPilot as a secure, tested modular monolith with Next.js Route Handlers, strict TypeScript, MongoDB, a separate worker for long-running jobs, and a provider-agnostic AI gateway. Establish the trusted financial data model and deterministic calculations before adding AI. Treat bank statement import as a versioned data pipeline, not just a file upload. Add bank connectivity only through a verified authorized path after eligibility, consent, security, and operational requirements are confirmed.

This TRD is a living engineering document. Update it when implementation choices change, and preserve a record of why the change was made. The code, tests, actual provider capabilities, and verified runtime behavior remain the source of truth; a document alone does not prove that a feature is implemented or secure.